K8s Assistant Lab

A hands-on Kubernetes troubleshooting lab. You launch five full-stack applications on Amazon EKS, backed by Amazon RDS for PostgreSQL, with images in ECR and public traffic entering through Route 53 and a shared Application Load Balancer. Then you break them on purpose and fix them.

Every exercise follows the same loop:

```
symptom -> scope -> Kubernetes or AWS evidence -> root cause -> fix -> verify recovery
```

Project walkthrough

Step	What you do	Where
1	Install the tools and check your AWS access	Prerequisites
2	Set your domain in the Terraform variables file	Configure the lab
3	Run one script that builds everything	Launch
4	Check the cluster and open the five apps	Verify
5	Break an app and troubleshoot it	Incident scenarios
6	Destroy everything when you are done	Destroy the lab

The lab creates billable AWS resources. Keep the teardown command at hand before you begin.

1. Check the prerequisites

- Bash 4.4 or newer (Linux, WSL, or macOS with a current Bash from Homebrew; the macOS system Bash 3.2 is too old)
- Terraform, AWS CLI, kubectl, Docker, Helm, jq, `envsubst`, `curl`, and `nslookup`
- An AWS account with credentials configured
- An existing Route 53 **public hosted zone**, with the domain's registrar NS records pointing at it
- Budget for real EKS, EC2, NAT Gateway, ALB, Route 53, and RDS resources

The resource names are fixed, so run one lab per AWS account.

Confirm your tools and AWS identity:

```
aws sts get-caller-identity
docker info
terraform version
kubectl version --client
helm version
```

2. Configure the lab

The only required setting is your domain:

```
cp terraform/terraform.tfvars.example terraform/terraform.tfvars
```

Open `terraform/terraform.tfvars` and set `dns_zone_name` to your Route 53 hosted zone, for example `dns_zone_name = "labs.example.org"`. The five apps are published as `<app>.<dns_zone_name>`.

Everything else has a default. To change the region, cluster name, node size or count, or database size, add the variable to `terraform.tfvars`. The full list is in Where to find the config.

3. Launch the lab

From the repository root:

```
./scripts/setup.sh
```

The script:

1. Runs preflight checks: tools, Docker, AWS login, `terraform.tfvars`, the hosted zone, and DNS delegation. It asks before continuing if the delegation looks wrong.
2. Finds or requests the `*.<domain>` ACM certificate.
3. Runs `terraform apply` for the VPC, EKS, ECR, RDS, IAM, and ALB controller role.
4. Points `kubectl` at the new cluster.
5. Builds and pushes ten images (a frontend and a backend per app) to ECR.
6. Creates the namespaces and access roles.
7. Creates one database and user per app on the shared RDS instance and stores the credentials as Kubernetes Secrets.
8. Deploys Redis for food-delivery and all five apps, then waits for every rollout to become healthy.
9. Installs metrics-server and the AWS Load Balancer Controller, applies the Ingresses, and creates the Route 53 records.
10. Prints the application URLs.

Expect it to take 20 minutes or more. It is safe to re-run: if a rollout fails, the script names the failing Deployment. Fix the cause and run it again.

4. Verify the deployment

```
aws eks update-kubeconfig --name dev-cluster --region us-east-1
kubectl get nodes
kubectl get deployments -A
kubectl get pods -A
kubectl get ingress -A
```

Every application Deployment should show all of its replicas Ready. The domain is saved to `.lab-domain`, so you can list the app URLs with:

```
for app in ecommerce banking food-delivery student-portal support-tickets; do
  echo "https://${app}.$(cat .lab-domain)"
done
```

Open each URL and confirm the app loads before you break anything. Commands that help you build a picture of a healthy app:

```
kubectl get all -n banking
kubectl get events -n banking
kubectl describe pod -n ecommerce -l app=ecommerce-backend
kubectl logs -n banking deploy/banking-backend
kubectl top pods -A
```

5. Work through the incident scenarios

Each scenario has a guide that starts from the symptom and a script under scripts/chaos/ that causes it. Run the script, then diagnose the problem using the guide without reading ahead to the answer.

Scenario	Trigger	Main skill
Bad Container Image Deployment	`./scripts/chaos/bad-deploy.sh <namespace> <deployment> <container>`	ECR and Kubernetes Deployments
Bad ConfigMap Rollout	`./scripts/chaos/break-config.sh <app>`	ConfigMaps and health probes
OOMKilled and Resource Limits	`./scripts/chaos/shrink-limits.sh <app>`	Kubernetes resource management
ALB and Ingress Routing Failure	`./scripts/chaos/break-ingress.sh <app>`	ALB and Kubernetes networking
High Application Latency	`./scripts/chaos/inject-latency.sh <app> [ms]`	Application performance and workload troubleshooting

`<app>` is one of `ecommerce`, `banking`, `food-delivery`, `student-portal`, or `support-tickets`.

To get back to a healthy state:

- `break-config.sh`, `shrink-limits.sh`, and `break-ingress.sh` take `--undo`.
- `./scripts/chaos/reset.sh <app>` clears injected latency and other in-app chaos.
- `bad-deploy.sh` prints the `kubectl` command that reverts it.

Production-style write-ups of the same incidents are in DevOps STAR Scenarios, and the runbooks cover the fixes step by step.

6. Destroy the lab

The lab costs money while it runs. Teardown deletes the Ingresses first so the controller can release the ALB, removes the controller and namespaces, and runs `terraform destroy`.

```
./scripts/teardown.sh
```

Afterward, check the AWS console for any remaining ALBs, target groups, NAT Gateways, Elastic IPs, ECR repositories, or RDS instances.

Where to find the config

What	File	Notes
Your lab settings (domain, region, cluster name, nodes, DB size)	`terraform/terraform.tfvars`	Created from terraform.tfvars.example. Gitignored.
All Terraform variables and their defaults	terraform/variables.tf	Defaults: `us-east-1`, `dev-cluster`, 3 Ã— `t3.medium`, Kubernetes 1.33, `db.t3.micro` Postgres 17
AWS infrastructure	terraform/	`vpc.tf`, `eks.tf`, `ecr.tf`, `rds.tf`, `dns.tf`, `acm.tf`, `alb-controller.tf`
App runtime settings	`apps/<app>/k8s/configmap.yaml`	One per app, for example apps/banking/k8s/configmap.yaml
App Deployments, Services, autoscaling	`apps/<app>/k8s/`	`deployment-*.yaml`, `service-*.yaml`, `hpa-backend.yaml`. `${ECR_REGISTRY}` and `${IMAGE_TAG}` are filled in by `setup.sh`.
Database schema and seed data	`apps/<app>/backend/sql/init.sql`	Applied by `setup.sh`
Database credentials	Kubernetes Secret `<app>-db-credentials` in each app namespace	Generated by `setup.sh`, never stored in Git
Hostname routing	ingress/	One Ingress per app on the shared ALB
Namespaces, quotas, limit ranges	namespaces/	
Cluster access roles	rbac/	Used by `scripts/users.sh`
Incident triggers	scripts/chaos/	See the scenario table
Launch, teardown, cluster sharing	scripts/setup.sh, scripts/teardown.sh, scripts/users.sh	

Files that `setup.sh` generates locally, all gitignored: `.lab-domain` (your lab domain), `terraform/*.tfstate` (includes the RDS master password), and `kubeconfigs/` (access files from `users.sh`).

Architecture

Five applications share the EKS cluster: `ecommerce`, `banking`, `food-delivery`, `student-portal`, and `support-tickets`. Each has a React/nginx frontend and a Node.js/Express backend in its own namespace. See

Architecture for the full design.

Repository layout

```
apps/                Application source and Kubernetes manifests
docs/                Guides, scenarios, runbooks, and PDFs
ingress/             One hostname-based Ingress per application
namespaces/          Namespaces, quotas, and limit ranges
rbac/                Cluster access roles and the per-user sandbox template
scripts/             Setup, teardown, cluster sharing, PDF generation, and chaos triggers
terraform/           VPC, EKS, ECR, RDS, Route 53, IAM, and ALB controller
```

Optional: share your cluster with teammates

Teammates can use your cluster without an AWS account. `users.sh` creates a kubeconfig file for them:

```
./scripts/users.sh add alex sam      # one file per person, each with a personal sandbox namespace
./scripts/users.sh add team --readonly  # or: one shared read-only file
./scripts/users.sh list
./scripts/users.sh remove alex       # revoke access
```

Files are written to `kubeconfigs/`. A teammate saves theirs outside any repository and points `kubectl` at it:

```
export KUBECONFIG=~/.kube/lab.kubeconfig          # Bash / WSL / macOS
```

```
$env:KUBECONFIG = "$HOME\.kube\lab.kubeconfig"    # PowerShell
```

Shared access is read-only on the five apps: teammates can view pods, logs, events, and Ingresses, but cannot read Secrets or change anything. With a personal file they also get a `sandbox-<name>` namespace to practice in, capped at about five small pods. `Forbidden` errors outside the sandbox are expected.

Keep these files safe:

- **Each file contains a login token.** Send it privately, never through Git or a public channel. Tokens do not expire. Revoke them with `remove`.
- **Sandboxes use cluster capacity.** Keep it to about 10 people on the default three nodes. If pods sit `Pending`, raise `node_group_desired_size` in `terraform/terraform.tfvars` and re-run `./scripts/setup.sh`.
- **The API endpoint and app URLs are public.** Anyone with a token can query the cluster, and anyone can call the apps' `/api/chaos/*` endpoints, so agree with your teammates on who triggers incidents.

Guides and PDFs

- Connecting to a shared lab cluster, also as PDF
- Student guide
- Scenario test checklist
- Runbooks
- DevOps STAR scenario PDF
- All PDFs